Event-driven Architecture

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

Learning Objectives

- Explain what event-driven architecture (EDA) is and how it differs from synchronous request-response communication.
- Identify the core components of an event-driven system: producers, consumers, channels, and brokers.
- Distinguish between message queues and publish-subscribe (pub/sub) models.
- Describe the key AWS services for event-driven communication: SNS, SQS, and EventBridge.
- Describe the key GCP services for event-driven communication: Pub/Sub and Eventarc.
- Evaluate the advantages and disadvantages of event-driven architecture in distributed cloud systems.
- Explain delivery guarantees (at-least-once, exactly-once), idempotency, dead letter queues, and visibility timeouts.
- Recognize common event-driven patterns: fan-out, fan-in, load balancing, and event-driven serverless workflows.

Why Event-driven Architecture?

The Problem with Synchronous Communication

- In a traditional web application, components talk to each other using synchronous request-response calls. Service A sends an HTTP request to Service B and waits — it blocks — until Service B responds. This is simple and intuitive, but it creates problems as systems grow.
- Consider a simple e-commerce checkout. When a user places an order, the system needs to charge their payment, update inventory, send a confirmation email, and notify the warehouse. In a synchronous design, the order service calls each of these downstream services one after another, waiting for each to respond before calling the next.
- This approach has several weaknesses that become severe at cloud scale.

```
SYNCHRONOUS REQUEST-RESPONSE
==============================

  User places order
        |
        v
  +----------------+    HTTP     +-------------------+
  | Order Service | ---------> | Payment Service  |
  |                | <--------- |                   |
  +----------------+   (waits)  +-------------------+
        |
        v  (only after payment responds)
  +----------------+    HTTP     +-------------------+
  | Order Service | ---------> | Inventory Service|
  |                | <--------- |                   |
  +----------------+   (waits)  +-------------------+
        |
        v  (only after inventory responds)
  +----------------+    HTTP     +-------------------+
  | Order Service | ---------> | Email Service    |
  |                | <--------- |                   |
  +----------------+   (waits)  +-------------------+
        |
        v
  Response to user (after ALL services respond)

  Total latency = Payment + Inventory + Email latencies combined
```

The Problem with Synchronous Communication

- **Temporal coupling** means the sender and receiver must both be running at the same time. If the email service is down for maintenance, the entire checkout fails — even though sending a confirmation email is not critical to completing the order.
- **Tight dependency chains** mean that every service must know the network address of every other service it needs to call. Adding a new downstream step (say, a fraud detection service) requires changing the order service code and redeploying it.
- **Cascading failures** occur when one slow or failing service causes all upstream callers to back up. If the inventory service is slow, the order service threads pile up waiting, and eventually the order service itself becomes unresponsive. This is one of the most common causes of outages in distributed systems.
- **Scalability bottlenecks** arise because the order service can only process new orders as fast as its slowest downstream dependency responds.

The Event-driven Alternative

- Event-driven architecture (EDA) inverts this model. Instead of the order service calling each downstream service directly, it simply publishes an event — a record that something happened — to a message broker. The broker is a managed intermediary service that receives, stores, and delivers events. Each downstream service subscribes to the events it cares about and processes them independently, at its own pace.
- The order service responds to the user immediately after publishing the event. Each downstream service processes the event on its own timeline. If the email service is temporarily down, the event waits in the broker until the service recovers — no data is lost, and no other service is affected.

```
EVENT-DRIVEN ARCHITECTURE
=========================

  User places order
        |
        v
  +---------------+     "OrderPlaced" event
  | Order Service | -------------------------+
  +---------------+                          |
        |                                    v
        v                            +-----------------+
  Response to user                   |  Message Broker  |
  (immediately!)                     |  (SNS / Pub/Sub) |
                                     +-----------------+
                                       |       |       |
                            +----------+   +---+   +---+----------+
                            v              v              v
                     +-----------+  +-----------+  +-----------+
                     |  Payment  |  | Inventory |  |   Email   |
                     |  Service  |  |  Service  |  |  Service  |
                     +-----------+  +-----------+  +-----------+

  - Order Service does NOT wait for downstream services
  - Each consumer processes events independently
  - If Email Service is down, events queue up and are processed later
  - Adding a new consumer requires ZERO changes to Order Service
```

What Is an Event?

- An event is a record that something happened in a system. It is not a command telling another service what to do — it is a notification that a fact occurred. This distinction matters: the producer does not know or care who will consume the event, or what they will do with it.
- An event typically contains a type or name (what happened), a timestamp (when it happened), a source (which component produced it), and a payload (relevant data about what happened). For example, an "OrderPlaced" event might carry the order ID, customer ID, item list, and total amount.

Types of Coupling That EDA Reduces

- **Spatial coupling (location):** In synchronous systems, Service A must know Service B's network address. In EDA, producers only know the broker's address. Consumers can be added, removed, or relocated without the producer knowing.
- **Temporal coupling (time):** In synchronous systems, both services must be online simultaneously. In EDA, the broker stores events until consumers are ready. This is critical in the cloud, where instances scale up and down, get replaced, and occasionally fail.
- **Behavioral coupling (logic):** In synchronous systems, the order service must contain logic for each downstream call. In EDA, the order service publishes a single event. New consumers subscribe independently without modifying the producer.

Core Concepts and Messaging Models

Producers, Consumers, Channels, and Brokers

Every event-driven system has four fundamental building blocks.

- **Producers** (also called publishers) are the components that detect that something happened and create an event. A web application backend, an IoT sensor, or a database change stream can all be producers.
- **Consumers** (also called subscribers) are the components that receive and process events. A consumer might update a database, send a notification, trigger a workflow, or feed data into an analytics pipeline.
- **Channels** are named destinations where events are sent. Depending on the service, these are called topics, queues, or event buses. A channel provides a logical separation — order events go to an "orders" channel, user events go to a "users" channel.
- **Brokers** are the managed infrastructure services that receive events from producers, store them durably, and deliver them to consumers. In cloud computing, you almost never run your own broker — you use a managed service like AWS SNS, AWS SQS, GCP Pub/Sub, or AWS EventBridge.

Two Messaging Models: Queues vs. Pub/Sub

- There are two fundamental models for delivering events, and understanding the difference is essential.
- **Message Queue (point-to-point):** A message is sent to a queue and delivered to exactly one consumer. If multiple consumers are reading from the same queue, each message goes to only one of them. This is ideal for distributing work across a pool of workers — load balancing. Think of it like a shared to-do list: once one person picks up a task, the others do not see it.
- **Publish-Subscribe (pub/sub):** A message is published to a topic and delivered to all subscribers. Every subscriber gets its own copy of every message. This is ideal for broadcasting events to multiple independent systems — fan-out. Think of it like a newspaper: every subscriber gets their own copy.

```
MESSAGE QUEUE (Point-to-Point)
==============================

  Producer                      Consumer A
     |          +-------+         ^
     +--------->| Queue |------>--+
                |       |------>--+
                +-------+         v
                               Consumer B

  Each message goes to ONE consumer only.
  Used for: load balancing, work distribution.
  AWS service: SQS
  GCP equivalent: (Pub/Sub with a single subscription
                   and multiple subscribers)


PUBLISH-SUBSCRIBE (Fan-out)
===============================

  Producer                    Consumer A (email)
     |         +-------+        ^
     +-------->| Topic |----->--+
               |       |----->--+----> Consumer B (analytics)
               +-------+----->--+
                                +----> Consumer C (audit log)

  Each message goes to ALL subscribers.
  Used for: broadcasting, fan-out, event notification.
  AWS service: SNS
  GCP service: Pub/Sub
```

AWS Event-driven Services

Amazon Simple Notification Service (SNS)

- Amazon SNS is a fully managed pub/sub messaging service. Producers publish messages to an SNS topic, and SNS delivers copies of that message to all subscribed endpoints.
- SNS supports multiple subscriber types — it is not limited to application-to-application communication. A single SNS topic can simultaneously deliver messages to SQS queues, Lambda functions, HTTP/HTTPS endpoints, email addresses, and SMS phone numbers. This makes SNS a versatile notification hub.
- **Key characteristics of SNS:** SNS is a push-based system. When a message arrives, SNS immediately pushes it to all subscribers. There is no polling involved. SNS does not retain messages — once a message is delivered (or delivery is attempted), it is gone. If a subscriber is offline when the message is sent, that message is lost unless the subscriber endpoint is an SQS queue (which stores the message durably).
- **Message durability in SNS:** When SNS receives a publish request, it stores multiple copies of the message across multiple Availability Zones within the region before confirming receipt. This protects against data loss from infrastructure failures during the brief delivery window.

SNS Topic Types

- ***Standard topics*** provide best-effort ordering and at-least-once delivery. They support a very high message throughput and are suitable for most use cases where strict ordering is not required.
- ***FIFO topics*** guarantee strict message ordering and exactly-once delivery. They are useful when consumers must process events in the exact sequence they were produced — for example, processing financial transactions in order. FIFO topics have lower throughput limits compared to standard topics.
- **Message filtering:** SNS supports subscription filter policies. Instead of every subscriber receiving every message, you can attach a filter policy to a subscription that tells SNS to deliver only messages whose attributes match certain criteria. For example, an "orders" topic might carry events for all order states, but the shipping service subscribes with a filter that only delivers events where order_status = "paid". This avoids unnecessary processing in consumers and reduces costs.

Amazon Simple Queue Service (SQS)

- Amazon SQS is a fully managed message queue service. Unlike SNS's push-based pub/sub model, SQS is a pull-based queue. Consumers poll the queue to retrieve messages, process them, and then delete them from the queue.
- **Visibility timeout:** This is one of the most important SQS concepts and a common source of confusion. When a consumer retrieves a message from the queue, SQS does not immediately delete it. Instead, SQS makes the message invisible to other consumers for a configurable period called the visibility timeout (default: 30 seconds). During this window, the consumer that retrieved the message is expected to process it and then explicitly delete it. If the consumer crashes or fails to delete the message before the visibility timeout expires, the message becomes visible again in the queue and another consumer can pick it up. This mechanism ensures that messages are not lost if a consumer fails.

SQS VISIBILITY TIMEOUT

```
SQS VISIBILITY TIMEOUT
========================

Time ------>

  [Message visible]  -->  Consumer retrieves message
                          |
                          v
  [Message INVISIBLE] --> Visibility timeout starts (e.g., 30s)
                          |
                          | Consumer processing...
                          |
        +----------------+----------------+
        |                                 |
   SUCCESS: Consumer                 FAILURE: Consumer crashes
   deletes message                   or timeout expires
        |                                 |
        v                                 v
   Message permanently             Message becomes VISIBLE
   removed from queue              again for other consumers
```

SQS

- **Standard queues** offer nearly unlimited throughput, at-least-once delivery (a message may occasionally be delivered more than once), and best-effort ordering (messages may arrive out of order). Standard queues are appropriate for workloads that can tolerate occasional duplicates and do not require strict ordering.
- **FIFO queues** guarantee exactly-once processing and strict first-in-first-out ordering. FIFO queues support up to 300 messages per second by default (or 3,000 per second with batching). They are appropriate for workloads where order matters — processing bank transactions, handling sequential workflow steps, or maintaining an ordered event log.
- **Dead letter queues (DLQ):** A dead letter queue is a secondary SQS queue that receives messages that could not be processed successfully after a configurable number of attempts (the maximum receive count). Without a DLQ, a poisoned message — one that consistently causes consumer failures — would cycle endlessly: it becomes visible, a consumer picks it up, the consumer fails, the visibility timeout expires, and the message becomes visible again. A DLQ breaks this cycle by moving the problematic message to a separate queue for inspection and debugging.

```
DEAD LETTER QUEUE FLOW
========================

  +-------------+    poll     +-----------+
  | Main Queue  | --------->  | Consumer  |
  +-------------+             +-----------+
        ^                           |
        |   (failure: msg becomes   |
        |    visible again)         |
        +---------------------------+

  After N failed attempts (maxReceiveCount):

  +-------------+    auto-move    +-----------+
  | Main Queue  | -------------> | Dead Letter|
  +-------------+                | Queue      |
                                 +-----------+
                                       |
                               Inspect, debug,
                               reprocess manually
```

Long polling vs. short polling

- By default, SQS uses **short polling** — when a consumer calls ReceiveMessage, SQS checks a subset of its servers and returns immediately, even if no messages are available. This can result in frequent empty responses.
- **Long polling** (enabled by setting the WaitTimeSeconds parameter to a value greater than 0, up to 20 seconds) causes SQS to wait until a message is available before responding. Long polling reduces empty responses, lowers API call costs, and reduces latency for message processing. In almost all cases, you should use long polling.

Pub/Sub Messaging Patterns

Patterns

```
FAN-IN                    FAN-OUT                  LOAD BALANCED
========                  =========                ==============

Producer A --+            Producer               Producer
Producer B --+--> Topic     |                       |
Producer C --+     |        v                       v
                   v      Topic                   Topic
              Subscription  |                       |
                   |        +--> Sub A --> Consumer  +--> Subscription
                   v        +--> Sub B --> Consumer    |
               Consumer     +--> Sub C --> Consumer    +--> Consumer A
                                                       +--> Consumer B
Aggregation          Broadcasting                      +--> Consumer C
from many            to many
sources              consumers                   Work distribution
```

Fan-in (Many-to-One)

- Multiple producers publish messages to a single topic, which delivers to a single subscription consumed by a single subscriber.
- This pattern is used to aggregate events from many sources into one processing pipeline.
- For example, hundreds of IoT devices publishing sensor readings to one topic, consumed by a single analytics pipeline.

Fan-out (One-to-Many)

- A single producer publishes messages to a topic that has multiple subscriptions, each consumed by a different service.
- Every consumer receives a copy of every message.
- This pattern is used when a single event needs to trigger multiple independent actions.
- For example, an "OrderPlaced" event fanning out to a payment processor, an inventory updater, an email sender, and an analytics recorder.

Load Balancing (Many-to-Many)

- Multiple producers publish to a topic that delivers to a single subscription, which is consumed by multiple subscriber instances.
- Each message goes to exactly one subscriber.
- This pattern is used to distribute work across a pool of identical workers for parallel processing.
- Adding more subscribers increases processing throughput.

Advantages and Disadvantages

Advantages of Event-driven Architecture

- **Loose coupling and independent deployability.** Producers and consumers do not know about each other. You can deploy, update, scale, or replace any consumer without affecting the producer or other consumers. In a cloud-native microservices environment, this is essential — teams can work independently and deploy on their own schedules.
- **Elastic scalability.** Message brokers act as a buffer between producers and consumers. When traffic spikes (a flash sale, a viral post, a batch upload), messages queue up in the broker rather than overwhelming downstream services. Consumers can autoscale based on queue depth — for example, an AWS Auto Scaling group can watch the ApproximateNumberOfMessagesVisible metric on an SQS queue and add more EC2 instances when the backlog grows.
- **Resilience and fault tolerance.** If a consumer service crashes or becomes unavailable, messages are retained in the broker until the service recovers. No events are lost. This contrasts sharply with synchronous systems where a downstream failure immediately causes upstream failures. In the cloud, where instances are ephemeral and failures are expected, this resilience is critical.
- **Natural fit for serverless.** Event-driven architecture pairs naturally with serverless compute. AWS Lambda functions can be triggered directly by SQS messages, SNS notifications, or EventBridge events. GCP Cloud Functions can be triggered by Pub/Sub messages. You pay only for the compute time spent processing each event — there is no idle infrastructure cost.
- **Extensibility without modifying existing code.** Adding a new consumer to react to an existing event requires zero changes to the producer. You simply create a new subscription or rule. This is particularly valuable in large organizations where different teams own different services.

Disadvantages of Event-driven Architecture

- **Eventual consistency.** In a synchronous system, after a request completes, the caller knows that all downstream operations have finished. In an event-driven system, the producer publishes an event and moves on — downstream processing happens later. This means the system is eventually consistent: there is a window of time where different components have different views of the system state. For example, after placing an order, the user's order history page might not show the order for a few seconds until the event has been processed. Designing user interfaces and business logic to handle eventual consistency requires careful thought.
- **Debugging and observability challenges.** In a synchronous system, you can trace a request from start to finish in a single call chain. In an event-driven system, a single user action might trigger a chain of events across many services, with each event processed asynchronously at unpredictable times. Tracing a problem through an event-driven system requires distributed tracing tools (AWS X-Ray, GCP Cloud Trace) and centralized logging. Without these, debugging production issues becomes extremely difficult.
- **Ordering and duplication complexity.** Most messaging systems guarantee at-least-once delivery, meaning a consumer might receive the same message more than once. Standard queues and standard SNS topics do not guarantee message ordering. Your consumer code must be designed to handle both duplicates and out-of-order messages gracefully. This is a significant development and testing burden that does not exist in simple synchronous systems.
- **Idempotency requirements.** Because messages can be delivered more than once (due to at-least-once delivery, network retries, or consumer failures), every consumer must be idempotent — processing the same message twice must produce the same result as processing it once. For example, a payment consumer that receives an "OrderPlaced" event twice must not charge the customer twice. Achieving idempotency requires careful design: deduplication tables, conditional writes, or idempotency keys.
- **Increased infrastructure complexity.** An event-driven system has more moving parts than a synchronous one. You need to provision and configure topics, queues, subscriptions, dead letter queues, filter policies, and retry policies. You need monitoring and alerting on queue depths, consumer lag, and error rates. The operational surface area is larger.
- **Message schema evolution.** When a producer changes the format of an event (adds a field, removes a field, changes a data type), all consumers must be able to handle both the old and new format without breaking. In a synchronous system with a shared API contract, versioning is more centralized. In an event-driven system with potentially many consumers owned by different teams, schema changes require coordination. Services like GCP Pub/Sub Schema and EventBridge Schema Registry help, but they add complexity.

Reliability Patterns and Best Practices

Idempotency in Practice

- Idempotent consumers produce the same result regardless of how many times they process the same message. Here are common approaches to achieving idempotency.
- **Idempotency keys:** Each event carries a unique identifier (often a UUID). The consumer stores processed IDs in a database. Before processing a new message, the consumer checks: have I already processed this ID? If yes, skip it. This is the most straightforward approach but requires a durable lookup store.
- **Conditional writes:** Instead of blindly inserting or updating, use conditional operations. For example, "UPDATE inventory SET count = count - 1 WHERE order_id != '123'" or "INSERT ... IF NOT EXISTS." The database enforces that the operation only takes effect once.
- **Natural idempotency:** Some operations are naturally idempotent. Setting a value (SET status = 'shipped') is idempotent — doing it twice has the same effect. Incrementing a counter (SET count = count - 1) is not idempotent — doing it twice subtracts two.

Retry Strategies and Backoff

- When a consumer fails to process a message, the message is redelivered. But retrying immediately and aggressively can make things worse — if the failure was caused by an overloaded downstream service, hammering it with retries will only make the overload worse.
- **Exponential backoff** is the standard retry strategy: wait 1 second, then 2, then 4, then 8, and so on, with an upper bound. This gives transient failures time to resolve.
- **Jitter** adds randomness to the backoff interval. Without jitter, hundreds of consumers that failed at the same time will all retry at the same time, creating a "thundering herd" that causes the same failure again.

Dead Letter Queues

- As discussed in the SQS section, dead letter queues catch messages that repeatedly fail processing. The key operational practice is to monitor your DLQs actively. A message in a DLQ represents a failure your system could not automatically recover from. DLQ monitoring should be part of your alerting setup — an alarm when the DLQ message count exceeds zero.
- Both AWS SQS and GCP Pub/Sub support dead letter queue / dead letter topic configurations. On GCP, a dead letter topic is a separate Pub/Sub topic that receives messages that could not be acknowledged after a configurable number of delivery attempts.

Choosing Delivery Guarantees

- **At-most-once delivery** means the message is delivered zero or one time. If anything goes wrong, the message is dropped. This is acceptable for non-critical data like metrics or logs where occasional data loss is tolerable.
- **At-least-once delivery** means the message is delivered one or more times. The system retries on failure, which may result in duplicates. This is the default for most cloud messaging services (standard SQS, standard SNS, Pub/Sub). Consumers must be idempotent.
- **Exactly-once processing** means the message is processed exactly once. This is the strongest guarantee and the hardest to achieve. FIFO SQS queues and FIFO SNS topics provide exactly-once semantics within the messaging layer. GCP Pub/Sub provides exactly-once delivery when using pull subscriptions with the exactly-once delivery feature enabled.

Cross-Cloud Comparison

Service Mapping Table

Service Mapping Table

Concept	AWS	GCP
Pub/Sub messaging (fan-out)	Amazon SNS	Cloud Pub/Sub
Message queue (point-to-point)	Amazon SQS	Cloud Pub/Sub (single subscription)
Event bus with content-based routing	Amazon EventBridge	Eventarc
Serverless event-driven compute	AWS Lambda (triggered by SNS/SQS/EventBridge)	Cloud Functions / Cloud Run (triggered by Pub/Sub/Eventarc)
FIFO / ordered messaging	SQS FIFO queues, SNS FIFO topics	Pub/Sub ordering keys
Dead letter queue / topic	SQS DLQ	Pub/Sub dead letter topic
Schema management	EventBridge Schema Registry	Pub/Sub Schema
Message filtering	SNS subscription filter policies	Pub/Sub subscription filters
Event standard	Custom JSON	CloudEvents specification

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS